CSCI 3060U - Phase 2: Front End Design Document

Digital Games Distribution Service · Arian Vares and Victor Ma · Due October 19, 2026

1. Overview

The Front End is a C++ console program. It reads transactions typed by a user (`login, logout, create, delete, sell, buy, refund, addcredit, list, listaccounts`), checks them against the user accounts file and the available games file, and writes a daily transaction file when the user logs out. It is started as `./frontend accounts.txt games.txt daily.txt`. The Back End (later phase) reads the daily transaction file.

The design has one class per transaction. They all share a base class **Transaction**, and a **TransactionFactory** turns the typed command into the right object. A **Session** object holds everything that changes while the program runs, so transactions don't need global variables. Files are only read by **RecordFile** (with **AccountRegistry** and **GameCatalog** checking each line) and only written by **DailyTransactionLog**. All terminal input and output goes through **Console**.

2. How the program runs

1. **main** reads the three file names and creates a **FrontEnd**.
2. **FrontEnd::run** prints `Enter transaction:` and reads a command until the input ends.
3. **handleCommand** asks the **TransactionFactory** for the transaction. No transaction means `ERROR: invalid transaction`. If it needs a login and nobody is logged in, `ERROR: please login first`. If someone is logged in but their account type can't use it, `ERROR: not allowed for this account type` (checked before any other prompt).
4. Otherwise **execute()** runs. It prompts for what it needs, checks the rules, and stops at the first error (nothing is recorded for a failed transaction).
5. A finished transaction updates the **Session** when needed and, if it needs a record (everything except login, list and listaccounts), adds a line to the **DailyTransactionLog**.
6. On **logout** (or an admin deleting their own account) **Session::end** adds the 00 line and writes the daily file. If the input ends without a logout, no daily file is written.

3. UML class diagrams

Figure 1 shows the control flow and the transaction classes. Figure 2 shows the data, file and helper classes. Symbols: + public, - private, # protected. Filled diamond = owns, open diamond = holds many, hollow triangle = inherits from, dashed arrow = uses / creates.

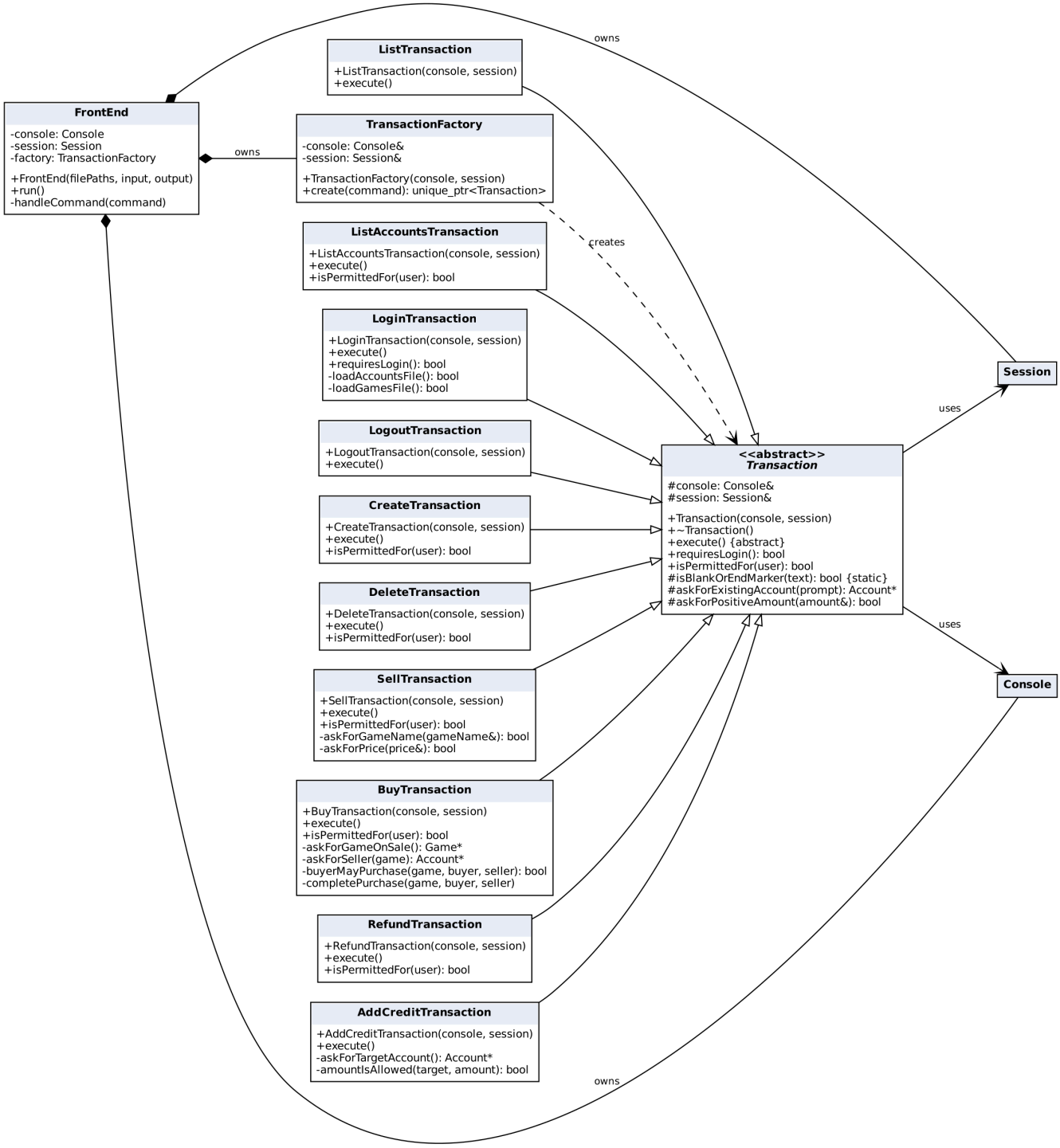


Figure 1 - FrontEnd, TransactionFactory and the Transaction classes

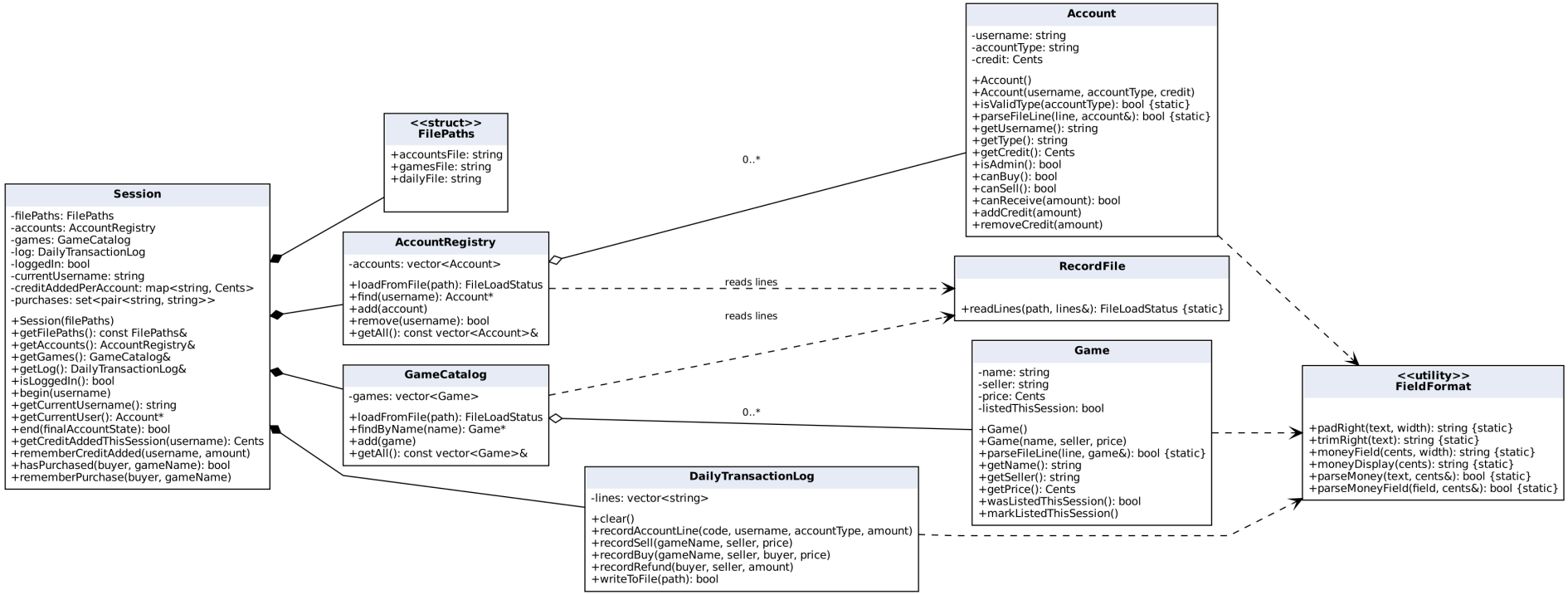


Figure 2 - Session, data, file and helper classes

4. Where the inputs and outputs fit

Input / output	Used by	What happens
Standard input (keyboard)	Console::readLine / ask, used by FrontEnd::run and every transaction	Commands and answers to prompts, one per line.
Standard output (terminal)	Console::show / showError	Prompts, confirmations, errors, and the output of list and listaccounts.
Standard error (stderr)	main	Prints the usage message if the program does not get exactly 3 file names and then exits with code 1.
User accounts file (argument 1)	LoginTransaction -> AccountRegistry::loadFromFile -> RecordFile::readLines -> Account::parseFileLine	Read at every login. 28 character lines ending at END. Missing file or bad line gives an error and the login fails.
Available games file (argument 2)	LoginTransaction -> GameCatalog::loadFromFile -> RecordFile::readLines -> Game::parseFileLine	Read at login after the username is checked. 48 character lines ending at END.
Daily transaction file (argument 3)	Session::end -> DailyTransactionLog::writeToFile	Written (replaced) when the session ends. Lines 00 to 06 as in the handout, the 00 line is last. If the file can not be written the program shows ERROR: cannot write daily transaction file.

File formats: account line = username (15) + space + type (2) + space + credit (9). Game line = name (25) + space + seller (15) + space + price (6). Daily lines: 00/01/02/06 use the account format with the code in front (06 holds the amount added), 03 = name, seller, price, 04 = name, seller, buyer, price, 05 = buyer, seller, amount (9 wide).

5. Transactions: required functionality

Command	Allowed	Input	Prompts	Errors	Success output	Daily line
`login`	everyone (not logged in yet)	accounts file, games file, stdin: username	Enter username:	already logged in; cannot open / bad format (accounts or games file); invalid username (blank or END); no such user	Login successful.	none
`logout`	everyone	stdin		please login first; cannot write daily transaction file	Logout successful. (only if the daily file was written)	00 + daily file written
`create`	AA	stdin: new username, user type	Enter new username: / Enter user type (AA, FS, BS, SS):	invalid username; username too long (max 15); username already exists; invalid user type	User created.	01 (credit 000000.00)

`delete`	AA	stdin: username	Enter username to delete:	invalid username; no such user; cannot write daily transaction file (only when an admin deletes themselves)	User deleted. (session ends if the admin deleted themselves)	02
`sell`	AA, FS, SS	stdin: game name, price	Enter game name: / Enter price:	invalid game name; game name too long (max 25); game already for sale; invalid price; price too high (max 999.99)	Game put up for sale.	03
`buy`	AA, FS, BS	stdin: game name, seller	Enter game name: / Enter seller username:	game not found; no such user; seller does not match the listing; game not available for purchase this session; cannot buy your own game; game already owned; not enough credit; seller credit limit exceeded	Game purchased.	04
`refund`	AA	stdin: buyer, seller, amount	Enter buyer username: / Enter seller username: / Enter amount of credit:	no such user; buyer and seller must be different; invalid amount; not enough credit; credit limit exceeded	Refund complete.	05
`addcredit`	everyone (AA also picks the account)	stdin: amount, username (AA only)	Enter amount of credit: / Enter username:	invalid amount; no such user; credit limit exceeded; session credit limit of 1000.00 exceeded	Credit added.	06 (amount added)
`list`	everyone	games list		none	Games for sale: then name | seller | price (or No games are currently for sale.)	none
`listaccounts`	AA	accounts list		none	Active accounts: then username | type | credit	none

Rules shared by all: money is kept as whole cents; the maximum credit is 999999.99; usernames are case sensitive and at most 15 characters; game names are at most 25 characters, case sensitive and unique across all sellers; a game listed this session can't be bought until a later session; a refund takes the money from the seller; addcredit is limited to 1000.00 per account per session.

6. Design decisions and assumptions

- **Reusing a deleted username:** the client said on Slack that a deleted username can be created again in the same session. The Front End allows this (a deleted account is removed from the registry, so create no longer sees it). This disagrees with our Phase 1 test `create_duplicate_deleted_this_session`, which expects "username already exists". That test needs to be looked at in Phase 3.
- **All-accounts transaction:** the handout doesn't name it. The instructor's Slack answer (Oct 17) says to use a unique, clear name for Phase 2 and get the client's required name by Phase 3, so we call it `listaccounts` for now. It is admin only (privileged) and prints one line per account: username | type | credit.
- **Front End files:** per the Oct 17 Slack answer the Front End never modifies the accounts or games files (that is the Back End's job); it only writes the daily transaction file.
- **Price 0.00** is accepted when selling (free games are allowed), and the **$1000.00 addcredit limit** applies to every addcredit, admin or not (Oct 17 Slack answers).
- Accounts and games are read from the files at every login (the files themselves are never changed by the Front End).
- Buy only knows about games bought during the current session (the Front End never receives the games collection file).
- Deleting a user does not remove the games they listed.
- An admin deleting themselves ends the session at once: delete (02) line, then the 00 line, no "Logout successful.".
- **Seller mismatch during buy:** if the entered seller has an account but is not the seller attached to that game listing, the Front End rejects the purchase with `ERROR: seller does not match the listing`. The Oct 17 Slack answer says a wrong seller name makes the transaction invalid.
- Permission errors show `ERROR: not allowed for this account type`.
- Money is stored as whole cents (the Cents type) so all money math is exact. It is only turned into dollars when it is shown or written to a file.
- Every message is typed once in Messages.h without the ERROR: part. Console showError adds ERROR: in front.
- If the program does not get exactly 3 file names it prints a usage message to standard error and stops before reading any input.

7. Not finished or tested yet (left for Phase 3)

This is the rapid first version. It builds and runs on simple inputs that we tried by hand, but we have not run our 74 requirements tests against it yet. Anything those tests find (output wording, edge cases around limits, the deleted-username conflict above, bad input in the middle of a transaction) will be fixed in Phase 3.

8. Team split

Arian Vares	**Victor Ma**

FrontEnd, TransactionFactory, Transaction, Session (with FilePaths), RecordFile, AccountRegistry, GameCatalog, DailyTransactionLog, LoginTransaction, SellTransaction, BuyTransaction, RefundTransaction, AddCreditTransaction. UML diagrams, sections 1 to 3.	Constants.h, Messages.h, FieldFormat, Console, Account, Game, LogoutTransaction, CreateTransaction, DeleteTransaction, ListTransaction, ListAccountsTransaction, main.cpp, Makefile. Sections 4 to 6 and 8 and my rows in section 9.

9. Class and method descriptions

FrontEnd class · FrontEnd.h / FrontEnd.cpp · Arian

Class / method	Intention
FrontEnd	Main controller that keeps asking for transactions and checks login and permission before running each one.
`+FrontEnd(filePaths, input, output)`	Sets up the console, the session and the transaction factory.
`+run()`	Loops printing the transaction prompt and handling each typed command until the input ends.
`-handleCommand(command)`	Makes the transaction for a command, checks it is valid, logged in and permitted, then executes it.

TransactionFactory class · TransactionFactory.h / .cpp · Arian

Class / method	Intention
TransactionFactory	Turns the typed command word into the matching Transaction object.
`+TransactionFactory(console, session)`	Keeps the console and session to hand to every transaction it makes.
`+create(command): unique_ptr<Transaction>`	Returns the transaction for the command word, or an empty pointer if the word is not a transaction.

Transaction abstract class · Transaction.h / .cpp · Arian

Class / method	Intention
Transaction	Base class for all transactions; defines the execute / login / permission interface and the helpers they share.
`+Transaction(console, session)`	Keeps the console and session so the transaction can use them.
`+~Transaction()`	Lets a transaction be deleted safely through a Transaction pointer.
`+execute() {abstract}`	Does the whole transaction: prompts, checks, changes to the session and the daily file line.
`+requiresLogin(): bool`	Says whether someone must be logged in first (true for everything except login).
`+isPermittedFor(user): bool`	Says whether this account type may use the transaction (true unless a subclass restricts it).
`#isBlankOrEndMarker(text): bool {static}`	True if the typed text is empty or is END, so it can't be used as a name.

`#askForExistingAccount(prompt): Account*`	Prompts for a username and returns that account, or shows 'no such user' and returns NULL.
`#askForPositiveAmount(amount&): bool`	Prompts for an amount of credit and returns false with an error if it isn't a number above zero.

LoginTransaction class · LoginTransaction.h / .cpp · Arian

Class / method	Intention
LoginTransaction	Logs a user in: loads both input files, checks the username and starts the session.
`+LoginTransaction(console, session)`	Makes a login transaction.
`+execute()`	Refuses a second login, loads the accounts file, asks for and checks the username, loads the games file and begins the session.
`+requiresLogin(): bool`	Returns false because login is used while nobody is logged in.
`-loadAccountsFile(): bool`	Loads the accounts file into the session and shows the right error if it is missing or badly formatted.
`-loadGamesFile(): bool`	Loads the games file into the session and shows the right error if it is missing or badly formatted.

LogoutTransaction class · LogoutTransaction.h / .cpp · Victor

Class / method	Intention
LogoutTransaction	Logs the user out, which writes the daily transaction file.
`+LogoutTransaction(console, session)`	Makes a logout transaction.
`+execute()`	Ends the session (adds the 00 line and writes the daily file) and prints the logout message only if the file was written, otherwise shows an error.

CreateTransaction class · CreateTransaction.h / .cpp · Victor

Class / method	Intention
CreateTransaction	Lets an admin create a new account with 0.00 credit.
`+CreateTransaction(console, session)`	Makes a create transaction.
`+execute()`	Asks for a new username and account type, validates them, adds the account and records the 01 line.
`+isPermittedFor(user): bool`	Allows admin accounts only.

DeleteTransaction class · DeleteTransaction.h / .cpp · Victor

Class / method	Intention
DeleteTransaction	Lets an admin delete an account, and ends the session if the admin deleted themselves.
`+DeleteTransaction(console, session)`	Makes a delete transaction.

`+execute()`	Asks which account to delete, removes it, records the 02 line and ends the session if it was the logged in admin.
`+isPermittedFor(user): bool`	Allows admin accounts only.

SellTransaction class · SellTransaction.h / .cpp · Arian

Class / method	Intention
SellTransaction	Puts a new game up for sale under the logged in user's name.
`+SellTransaction(console, session)`	Makes a sell transaction.
`+execute()`	Gets a valid game name and price, adds the game as a new listing for this session and records the 03 line.
`+isPermittedFor(user): bool`	Allows every account type except buy-standard.
`-askForGameName(gameName&): bool`	Prompts for the game name and checks it is not blank, not END, not over 25 characters and not already for sale.
`-askForPrice(price&): bool`	Prompts for the price and checks it is a valid amount that is not over 999.99.

BuyTransaction class · BuyTransaction.h / .cpp · Arian

Class / method	Intention
BuyTransaction	Lets the logged in user buy a game, moving the price from the buyer's credit to the seller's.
`+BuyTransaction(console, session)`	Makes a buy transaction.
`+execute()`	Asks for the game and seller, checks the purchase is allowed and completes it.
`+isPermittedFor(user): bool`	Allows every account type except sell-standard.
`-askForGameOnSale(): Game*`	Prompts for the game name and finds it, or shows 'game not found'.
`-askForSeller(game): Account*`	Prompts for the seller's username and checks the seller exists and is the one selling that game.
`-buyerMayPurchase(game, buyer, seller): bool`	Checks the purchase rules in order (new listing, own game, already owned, buyer credit, seller credit limit) and shows the first error.
`-completePurchase(game, buyer, seller)`	Moves the credit, remembers the purchase and records the 04 line.

RefundTransaction class · RefundTransaction.h / .cpp · Arian

Class / method	Intention
RefundTransaction	Lets an admin move credit from a seller back to a buyer.
`+RefundTransaction(console, session)`	Makes a refund transaction.
`+execute()`	Asks for buyer, seller and amount, checks credit and limits, moves the credit and records the 05 line.
`+isPermittedFor(user): bool`	Allows admin accounts only.

AddCreditTransaction class · AddCreditTransaction.h / .cpp · Arian

Class / method	Intention
AddCreditTransaction	Adds credit to an account, with a 1000.00 limit per account per session.
`+AddCreditTransaction(console, session)`	Makes an addcredit transaction.
`+execute()`	Asks for the amount (and the username for admins), checks the limits, adds the credit and records the 06 line.
`-askForTargetAccount(): Account*`	Returns the logged in user for standard accounts, or asks an admin which existing account gets the credit.
`-amountIsAllowed(target, amount): bool`	Checks the account stays under the maximum credit and the session limit, and shows an error if not.

ListTransaction class · ListTransaction.h / .cpp · Victor

Class / method	Intention
ListTransaction	Prints every game currently for sale (Lab 1 update).
`+ListTransaction(console, session)`	Makes a list transaction.
`+execute()`	Prints each game as name | seller | price, or a message when nothing is for sale.

ListAccountsTransaction class · ListAccountsTransaction.h / .cpp · Victor

Class / method	Intention
ListAccountsTransaction	Prints every active account; the admin only transaction from the Lab 2 update.
`+ListAccountsTransaction(console, session)`	Makes a listaccounts transaction.
`+execute()`	Prints each account as username | type | credit.
`+isPermittedFor(user): bool`	Allows admin accounts only.

Session class · Session.h / .cpp · Arian

Class / method	Intention
Session	Holds everything the front end knows while running: who is logged in, loaded accounts and games, the daily file lines, credit added per account and purchases.
`+Session(filePaths)`	Starts with nobody logged in and keeps the file names.
`+getFilePaths(): const FilePaths&`	Returns the three file names.
`+getAccounts(): AccountRegistry&`	Returns the accounts for this session.
`+getGames(): GameCatalog&`	Returns the games for sale for this session.
`+getLog(): DailyTransactionLog&`	Returns the daily transaction lines collected so far.
`+isLoggedIn(): bool`	True while someone is logged in.

`+begin(username)`	Starts a session for the user and clears the log, credit totals and purchases from the last one.
`+getCurrentUsername(): string`	Returns the username of whoever is logged in.
`+getCurrentUser(): Account*`	Returns the logged in user's account, or NULL if nobody is logged in.
`+end(finalAccountState): bool`	Adds the 00 line, writes the daily transaction file and logs the user out.
`+getCreditAddedThisSession(username): Cents`	Returns how much credit was added to that account this session.
`+rememberCreditAdded(username, amount)`	Adds to the credit total for that account this session.
`+hasPurchased(buyer, gameName): bool`	True if that buyer already bought that game this session.
`+rememberPurchase(buyer, gameName)`	Remembers that the buyer bought the game.

FilePaths struct · Session.h · Arian

Class / method	Intention
FilePaths	Holds the three file names given on the command line.

AccountRegistry class · AccountRegistry.h / .cpp · Arian

Class / method	Intention
AccountRegistry	All the accounts that exist during a session, loaded from the user accounts file.
`+loadFromFile(path): FileLoadStatus`	Replaces the accounts in memory with the ones in the accounts file and reports if it can't be opened or has a bad line.
`+find(username): Account*`	Returns the account with that exact username or NULL.
`+add(account)`	Adds a new account.
`+remove(username): bool`	Removes the account with that username and returns false if there was none.
`+getAll(): const vector<Account>&`	Returns every account in file order.

GameCatalog class · GameCatalog.h / .cpp · Arian

Class / method	Intention
GameCatalog	All the games for sale during a session, loaded from the available games file.
`+loadFromFile(path): FileLoadStatus`	Replaces the games in memory with the ones in the games file and reports if it can't be opened or has a bad line.
`+findByName(name): Game*`	Returns the game with that exact name or NULL.
`+add(game)`	Adds a game to the end of the list.
`+getAll(): const vector<Game>&`	Returns every game in the order loaded or listed.

RecordFile class · RecordFile.h / .cpp · Arian

Class / method	Intention

RecordFile	Reads the lines of a fixed width input file up to its END line; also defines the FileLoadStatus result.
`+readLines(path, lines&): FileLoadStatus {static}`	Opens the file and collects every line before END, or reports it can't be opened or has no END line.

DailyTransactionLog class · DailyTransactionLog.h / .cpp · Arian

Class / method	Intention
DailyTransactionLog	Collects the lines of the daily transaction file for finished transactions and writes them out at the end of the session.
`+clear()`	Forgets all lines when a new session starts.
`+recordAccountLine(code, username, accountType, amount)`	Adds a logout (00), create (01), delete (02) or addcredit (06) line.
`+recordSell(gameName, seller, price)`	Adds a sell (03) line.
`+recordBuy(gameName, seller, buyer, price)`	Adds a buy (04) line.
`+recordRefund(buyer, seller, amount)`	Adds a refund (05) line.
`+writeToFile(path): bool`	Writes all the lines to the daily transaction file, replacing it, and returns false if it can't be written.

Account class · Account.h / .cpp · Victor

Class / method	Intention
Account	One user account with its type and credit, and what that type is allowed to do.
`+Account()`	Makes an empty account so parseFileLine has something to fill in.
`+Account(username, accountType, credit)`	Makes an account with these values.
`+isValidType(accountType): bool {static}`	True if the text is AA, FS, BS or SS.
`+parseFileLine(line, account&): bool {static}`	Reads one 28 character accounts file line into an account and returns false if the format is wrong.
`+getUsername(): string`	Returns the username.
`+getType(): string`	Returns the account type.
`+getCredit(): Cents`	Returns the credit in cents.
`+isAdmin(): bool`	True for admin accounts.
`+canBuy(): bool`	True unless the account is sell-standard.
`+canSell(): bool`	True unless the account is buy-standard.
`+canReceive(amount): bool`	True if adding the amount keeps the credit at or under 999,999.99.
`+addCredit(amount)`	Adds the amount to the credit.
`+removeCredit(amount)`	Takes the amount away from the credit.

Game class · Game.h / .cpp · Victor

Class / method	Intention
Game	One game for sale with its name, seller and price, and whether it was listed this session.
`+Game()`	Makes an empty game so parseFileLine has something to fill in.
`+Game(name, seller, price)`	Makes a game with these values.
`+parseFileLine(line, game&): bool {static}`	Reads one 48 character games file line into a game and returns false if the format is wrong.
`+getName(): string`	Returns the game name.
`+getSeller(): string`	Returns the seller's username.
`+getPrice(): Cents`	Returns the price in cents.
`+wasListedThisSession(): bool`	True if the game was put up for sale during this session.
`+markListedThisSession()`	Marks the game as a new listing for this session.

Console class · Console.h / .cpp · Victor

Class / method	Intention
Console	All terminal input and output, so no other class touches cin or cout.
`+Console(input, output)`	Remembers which streams to use.
`+show(text)`	Prints one line.
`+showError(message)`	Prints a line with ERROR: in front of the message.
`+readLine(line&): bool`	Reads one typed line (removing a trailing \r) and returns false when the input has ended.
`+ask(prompt, answer&): bool`	Prints a prompt and reads the answer, returning false when the input has ended.

FieldFormat utility class · FieldFormat.h / .cpp · Victor

Class / method	Intention
FieldFormat	Static helpers that convert text and money to and from the fixed width file fields and typed input.
`+padRight(text, width): string {static}`	Adds spaces to the end of text up to the width.
`+trimRight(text): string {static}`	Removes the spaces at the end of text.
`+moneyField(cents, width): string {static}`	Formats cents as a zero padded field such as 015.00.
`+moneyDisplay(cents): string {static}`	Formats cents as plain terminal text such as 15.00.
`+parseMoney(text, cents&): bool {static}`	Reads typed money like 15 or 15.5 into cents and rejects anything else.

`+parseMoneyField(field, cents&): bool {static}`	Reads a money field from a file and rejects it if the format is wrong.

Constants.h / Messages.h constants · Constants.h, Messages.h · Victor

Class / method	Intention
Constants.h / Messages.h	Not classes. Constants holds every limit, field width, keyword and transaction code. Messages holds every prompt, confirmation and error text (errors without ERROR: in front). Cents is the type used for money in whole cents. Each value is typed only once.

main function · main.cpp · Victor

Class / method	Intention
main	Program entry point.
`main(argc, argv): int`	Checks there are exactly 3 file names (prints a usage message to standard error if not), puts them in FilePaths, then makes the FrontEnd and runs it.